

```

0xA9, 0x00,      ; LDA #$00      ; Load 0 into the accumulator
0x8D, 0x40, 0x00, ; STA $0040     ; Store first Fibonacci number (0) in $0040
0xA9, 0x01,      ; LDA #$01      ; Load 1 into the accumulator
0x8D, 0x41, 0x00, ; STA $0041     ; Store second Fibonacci number (1) in $0041

```

```

0xA9, 0x02,      ; LDA #$02      ; Initialize counter to 2
0x8D, 0x30, 0x00, ; STA $0030     ; Store counter at $0030

```

LOOP:

```

0xAD, 0x41, 0x00, ; LDA $0041     ; Load F(n-1) into A
0xA8,             ; TAY           ; Transfer F(n-1) to Y for later
0xAD, 0x40, 0x00, ; LDA $0040     ; Load F(n-2) into A
0x6D, 0x41, 0x00, ; ADC $0041     ; Add F(n-1) to F(n-2)
0x8D, 0x42, 0x00, ; STA $0042     ; Store new Fibonacci number in $0042

```

```

0x98,             ; TYA           ; Transfer Y (old F(n-1)) back to A
0x8D, 0x40, 0x00, ; STA $0040     ; Update F(n-2) to previous F(n-1)
0xAD, 0x42, 0x00, ; LDA $0042     ; Load new Fibonacci number
0x8D, 0x41, 0x00, ; STA $0041     ; Update F(n-1) to current term

```

```

0xAD, 0x30, 0x00, ; LDA $0030     ; Load counter
0x69, 0x01,      ; ADC #$01      ; Increment counter by 1
0x8D, 0x30, 0x00, ; STA $0030     ; Store updated counter

```

```

0xC9, 0x0A,      ; CMP #$0A      ; Compare counter with 10
0xD0, 0xE6,      ; BNE loop      ; If counter != 10, branch back to loop

```

```

          ; end:
0x00      ; BRK          ; End program

```

```

0xA9, 0x00, 0x8D, 0x40, 0x00, 0xA9, 0x01, 0x8D, 0x41, 0x00, 0xA9, 0x02, 0x8D, 0x30, 0x00,
0xAD, 0x41, 0x00, 0xA8, 0xAD, 0x40, 0x00, 0x6D, 0x41, 0x00, 0x8D, 0x42, 0x00, 0x98, 0x8D,
0x40, 0x00, 0xAD, 0x42, 0x00, 0x8D, 0x41, 0x00, 0xAD, 0x30, 0x00, 0x69, 0x01, 0x8D, 0x30,
0x00, 0xC9, 0x0A, 0xD0, 0xE6, 0x00

```